

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

A) Create a file by using

Then press enter and the code terminal will open then write a code

```
balaji@BalajiPatil:~$ nano shell.c
```

```
GNU nano 7.2                                shell.c

#include <stdio.h>
#include <string.h>

int main() {
    char input[100];

    while (1) {
        printf("myShell> ");
        fgets(input, sizeof(input), stdin);

        // Remove newline
        input[strcspn(input, "\n")] = 0;

        // Exit condition
        if (strcmp(input, "exit") == 0) {
            printf("Exiting Shell...\n");
            break;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

After writing the code save the your program by ctrl O and for coming back to command press ctrl X then compile it by the command

```
balaji@BalajiPatil: ~  
balaji@BalajiPatil:~$ nano shell.c  
balaji@BalajiPatil:~$ gcc shell.c -o shell  
  
balaji@BalajiPatil:~$ nano shell.c  
balaji@BalajiPatil:~$ gcc shell.c -o shell  
balaji@BalajiPatil:~$ ./shell  
myShell> HI  
You entered: HI  
myShell> I am learning OSCP  
You entered: I am learning OSCP  
myShell> This is my first programming  
You entered: This is my first programming  
myShell> 
```

After compiling to get the output enter command

Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. **Open()**
 2. **Read()**
 3. **Write()**
 4. **Close()**
- open()**

syntax: `open("filename.txt", modes, permissions)`

Example: `open("First_Prgm.txt", O_CREAT | O_WRONLY, 0644);`

Table 1: Modes of Open system call

Flag	Purpose
O_RDONLY	Open for read only
O_WRONLY	Open for write only
O_RDWR	Open for both read and write
O_CREAT	Create file if it does not exist
O_TRUNC	Truncate (empty) an existing file
O_APPEND	Append data to the end of the file
O_EXCL	Fail if file already exists (used with O_CREAT)

read()

Syntax: read(int fd, void *buf, size_t count);

Example: read(fd, buffer, 100)

#include <stdio.h>

#include <unistd.h>

int main()

```
{
    char buffer[100];
    int n;
    printf("Enter some text: ");
    n = read(0, buffer, sizeof(buffer) - 1);
    buffer[n] = '\0'; // Null-terminate the string
    printf("You entered: %s", buffer);
    return 0;
}
```

write()

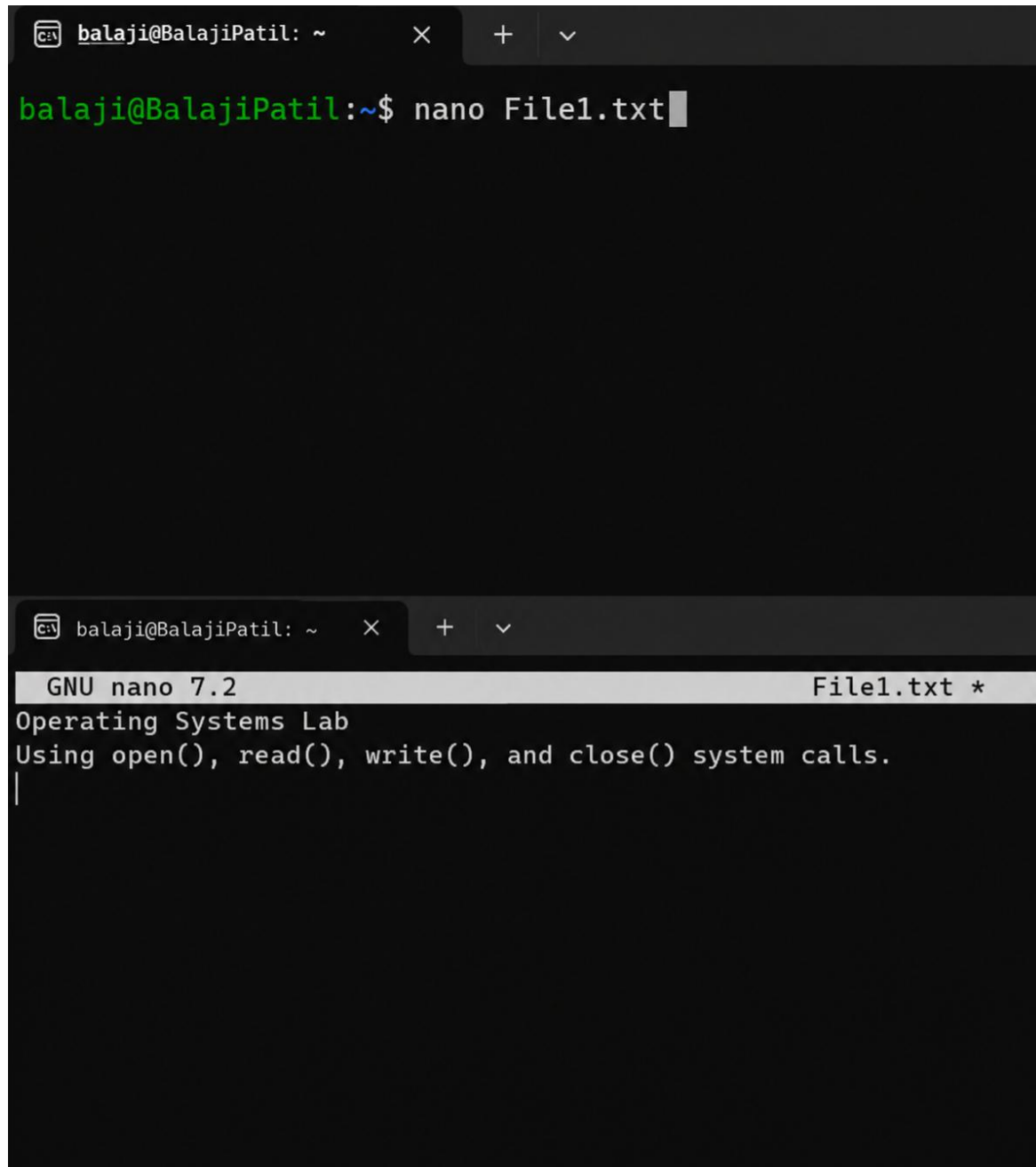
Syntax: write(int fd, const void *buf, size_t count);

Example: write(fd, message, 11);

#include <unistd.h>

int main()

```
{
    write(1, "Hello World\n", 12);
    return 0;
}
```



The image shows a terminal window with a dark background. At the top, a tab bar displays 'balaji@BalajiPatil: ~' with a close button (X), a plus sign (+), and a dropdown arrow (v). Below the tab bar, the command prompt 'balaji@BalajiPatil:~\$' is followed by 'nano File1.txt'. The terminal then transitions to the nano text editor interface. A status bar at the top of the editor shows 'GNU nano 7.2' on the left and 'File1.txt *' on the right. The main editing area contains two lines of text: 'Operating Systems Lab' and 'Using open(), read(), write(), and close() system calls.', followed by a blank line with a cursor at the start.

```
balaji@BalajiPatil: ~  
balaji@BalajiPatil:~$ nano File1.txt  
  
GNU nano 7.2 File1.txt *  
Operating Systems Lab  
Using open(), read(), write(), and close() system calls.  
|
```

```
balaji@BalajiPatil: ~  
balaji@BalajiPatil:~$ nano File1.txt  
balaji@BalajiPatil:~$ nano copy.c  
balaji@BalajiPatil:~$  
  
GNU nano 7.2 copy.c  
#include <stdio.h>  
#include <fcntl.h>  
#include <unistd.h>  
#include <stdlib.h>  
  
int main()  
{  
    int source, destination;  
    char buffer[1024];  
    ssize_t bytesRead;  
  
    // Open File1.txt for reading  
    source = open("File1.txt", O_RDONLY);  
  
    if (source < 0)  
    {  
        printf("Error opening File1.txt\n");  
        return 1;  
    }  
  
    // Open File2.txt for writing (create if it doesn't exist)  
    destination = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);  
  
    if (destination < 0)  
    {  
        printf("Error opening File2.txt\n");  
        return 1;  
    }  
  
    [ Read 44 lines ]  
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^J Execute   ^C Location  M-U Undo  
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^T Justify   ^_ Go To Line M-E Redo
```

balaji@BalajiPatil: ~ × + ▾

```
balaji@BalajiPatil:~$ nano File1.txt
```

```
balaji@BalajiPatil:~$ nano copy.c
```

```
balaji@BalajiPatil:~$ gcc copy.c -o copy █
```

balaji@BalajiPatil: ~ × + ▾

```
balaji@BalajiPatil:~$ nano File1.txt
```

```
balaji@BalajiPatil:~$ nano copy.c
```

```
balaji@BalajiPatil:~$ gcc copy.c -o copy
```

```
balaji@BalajiPatil:~$ ./copy █
```

```
balaji@BalajiPatil: ~  
balaji@BalajiPatil: ~$ nano File1.txt  
balaji@BalajiPatil: ~$ nano copy.c  
balaji@BalajiPatil: ~$ gcc copy.c -o copy  
balaji@BalajiPatil: ~$ ./copy  
Content copied successfully.  
balaji@BalajiPatil: ~$ cat File1.txt
```

```
balaji@BalajiPatil: ~  
balaji@BalajiPatil: ~$ nano File1.txt  
balaji@BalajiPatil: ~$ nano copy.c  
balaji@BalajiPatil: ~$ gcc copy.c -o copy  
balaji@BalajiPatil: ~$ ./copy  
Content copied successfully.  
balaji@BalajiPatil: ~$ cat File1.txt  
Operating Systems Lab  
Using open(), read(), write(), and close() system calls.  
balaji@BalajiPatil: ~$ cat File2.txt  
Operating Systems Lab  
Using open(), read(), write(), and close() system calls.  
balaji@BalajiPatil: ~$
```